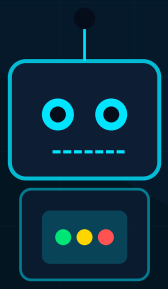
MACHINE LEARNING • FACULTY PROJECT

House Price Prediction

Regression Analysis & MLflow Tracking



Elastic Net Regression • Cross Validation • MLflow Experiment Tracking

PREPARED BY

1. Abdullah Waleed Kamal Mousa Salem — ID: 20230192
2. Mohamed Atef Mohamed Ali Shaheen — ID: 20230431
3. Saif Sameh Fathy Elsawy — ID: 20230142

Python | Scikit-Learn | MLflow | ElasticNet | Pandas | Matplotlib | NumPy

Table of Contents

Documentation Overview

01 Project Overview

Problem Statement & Objectives

PROBLEM STATEMENT

Predict residential property sale prices using historical housing data. The model uses Elastic Net Regression — a regularized linear approach combining L1 (Lasso) and L2 (Ridge) penalties to handle multicollinearity and perform automatic feature selection.

Regression Modeling

Apply Elastic Net to predict continuous house prices

Hyperparameter Tuning

Use GridSearchCV for optimal alpha and l1_ratio values

Experiment Tracking

Log all runs, params, and metrics via MLflow

TECHNOLOGY STACK

Python 3.x	Scikit-Learn	MLflow	Pandas	NumPy	Matplotlib
Core Language	ML Framework	Experiment Tracking	Data Manipulation	Numerical Computing	Visualization

MLFLOW WORKFLOW

Load Data → Preprocess → Train Model → Log to MLflow → Evaluate → Compare Runs

02 Dataset & Features

Housing Data Attributes & Structure

Domain	Target	Task Type	Model
Real Estate Housing Market	**Sale Price (USD) Continuous Value**	**Supervised Regression**	**Elastic Net Regularization**

FEATURE CATEGORIES

Structural

- Overall Quality (1-10)
- Above Ground Sq Ft
- Total Basement Sq Ft
- Garage Area (sq ft)
- Year Built
- Year Remodeled

Location

- Neighborhood Category
- Lot Area (sq ft)
- Lot Frontage (ft)
- Street Type
- Land Contour
- Utilities Available

Interior

- Number of Bedrooms
- Number of Bathrooms
- Full/Half Baths
- Kitchen Quality
- Fireplace Count
- Basement Condition

External

- Exterior Material 1&2
- Roof Style/Material
- Foundation Type
- Porch Areas (SF)
- Pool Area
- Fence Quality

TARGET VARIABLE — SALE PRICE

- Continuous numerical value representing final sale price of residential property in USD
- Highly skewed distribution — log transformation often applied for better model performance
- Typical range: $35,000 – $755,000 | Mean ≈ $180,000 | Median ≈ $163,000

80%	20%	K
Training Set	**Test Set**	**CV Folds**
Model fitting & cross-validation	Final unbiased evaluation	GridSearchCV — logs as param: cv

04 Model Architecture

Elastic Net Regression — L1 + L2 Regularization

ELASTIC NET OBJECTIVE FUNCTION

$$\min \ (1/2n)||y - Xw||_2 + \text{alpha} * \text{l1\_ratio} * ||w||_1 + 0.5 * \text{alpha} * (1 - \text{l1\_ratio}) * ||w||_2$$

L1 — Lasso Component

- Drives coefficients to exactly zero
- Automatic feature selection
- Sparse solution — fewer features
- Useful when many features irrelevant

L2 — Ridge Component

- Shrinks coefficients towards zero
- Handles multicollinearity well
- All features kept (dense solution)
- Stable when features are correlated

KEY HYPERPARAMETERS (tracked in MLflow params)

alpha — Overall regularization strength. Higher = more regularization = simpler model. Searched via GridSearchCV, best value logged as best_alpha.

l1_ratio — Mix between L1 and L2. 0 = pure Ridge, 1 = pure Lasso, 0.5 = equal blend. Best value logged as best_l1_ratio in MLflow.

cv — Number of cross-validation folds used in GridSearchCV. Logged as param in MLflow for full reproducibility.

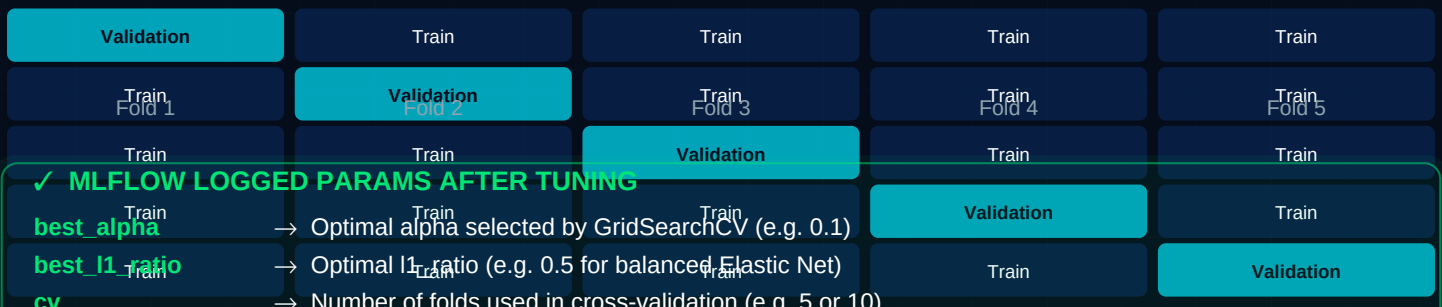
model — Model type identifier logged as string. Allows comparing different model architectures in the same MLflow experiment.

GridSearchCV — Exhaustive Search Strategy

GridSearchCV evaluates every combination of alpha and l1_ratio values using K-fold cross-validation. For each combination, it trains on (K-1) folds and validates on the remaining fold. The best combination minimises the CV error.

SEARCH GRID VISUALISATION

CROSS-VALIDATION PROCESS (K-Fold)



✓ MLFLOW LOGGED PARAMS AFTER TUNING

- **best_alpha** → Optimal alpha selected by GridSearchCV (e.g. 0.1)
- **best_l1_ratio** → Optimal l1_ratio (e.g. 0.5 for balanced Elastic Net)
- **cv** → Number of folds used in cross-validation (e.g. 5 or 10)
- **model** → Model name string (e.g. "ElasticNet")

06 MLflow Experiment Tracking

Params · Metrics · Tags — Full Run Structure

■ WHAT IS MLFLOW?

MLflow is an open-source platform for managing the entire ML lifecycle. It provides experiment tracking (logging params, metrics, artifacts per run), model versioning, and reproducibility tools. Every training run is recorded with its hyperparameters and results for easy comparison and auditing.

PARAMS (4 items)

best_alpha
Optimal regularization strength

best_l1_ratio
L1 vs L2 mix ratio

cv
CV folds count

model
Model type identifier

METRICS (6 items)

test_mae
Test Mean Abs Error

test_rmse
Test Root Mean Sq Err

test_r2
Test R-squared

train_mae
Train Mean Abs Error

train_rmse
Train Root Mean Sq Err

train_r2
Train R-squared

TAGS (4 items)

mlflow.runName
Descriptive run identifier

mlflow.source.name
Script/notebook that ran

mlflow.source.type
Source type (local/cloud)

mlflow.user
Username of experimenter

MLFLOW RUN LIFECYCLE

mlflow .start_run() - Log Params - Train Model - Log Metrics - Save Artifacts - mlflow .end_run()

WHY MLFLOW FOR THIS PROJECT?

- ■ Reproducibility → Every run is fully reproducible with exact params
- ■ Comparison → Compare multiple experiments side-by-side in the UI
- ■■ Organisation → All runs stored with timestamps, metrics, and artifacts
- ■ Deployment → Best model easily exported for production serving

07 Evaluation Metrics

MAE · RMSE · R^2 — Train & Test Performance

MAE

Mean Absolute Error

Average of absolute differences between predicted and actual prices. Easy to interpret: the average prediction error in dollars. Less sensitive to outliers than RMSE.

`train_mae` | `test_mae`

RMSE

Root Mean Square Error

Square root of the average squared errors. Penalises large errors more heavily than MAE. In same units as target (USD). Useful for detecting large prediction mistakes.

`train_rmse` | `test_rmse`

R^2

Coefficient of Determination

Proportion of variance in prices explained by the model. Range: 0 to 1 (1 = perfect fit). Answers: how well does the model explain price variability vs a baseline mean?

`train_r2` | `test_r2`

FORMULAS

MAE

$$MAE = (1/n) * sum(|y\_actual - y\_predicted|)$$

RMSE

$$RMSE = sqrt((1/n) * sum((y\_actual - y\_predicted)^2))$$

R^2

$$R^2 = 1 - (SS\_residual / SS\_total) = 1 - sum((y-y\_hat)^2) / sum((y-y\_mean)^2)$$

INTERPRETATION GUIDE

MAE & RMSE: Lower is better. RMSE > MAE indicates presence of large prediction errors.
R^2: Closer to 1.0 is better. R^2 > 0.85 is generally considered a strong fit for housing data.
Overfitting check: If train metrics >> test metrics, the model has overfit the training data.
Elastic Net regularization directly reduces overfitting by penalising model complexity.

09 Code Walkthrough

Annotated Step-by-Step Implementation

STEP 1 — Imports & Setup

```
import pandas as pd
import numpy as np
from sklearn.linear_model import ElasticNet
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import mlflow
import mlflow.sklearn
```

STEP 2 — Load & Split Data

```
df = pd.read_csv("housing_data.csv")
X = df.drop(columns=["SalePrice"])
y = df["SalePrice"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

STEP 3 — GridSearchCV Tuning

```
param_grid = {
    "alpha":    [0.001, 0.01, 0.1, 1, 10, 100],
    "l1_ratio": [0.1, 0.3, 0.5, 0.7, 0.9]
}
cv_model = GridSearchCV(ElasticNet(), param_grid, cv=5)
cv_model.fit(X_train, y_train)
```

STEP 4 — MLflow Logging

```
with mlflow.start_run():
    best_alpha    = cv_model.best_params_["alpha"]
    best_l1_ratio = cv_model.best_params_["l1_ratio"]
    mlflow.log_param("best_alpha",    best_alpha)
    mlflow.log_param("best_l1_ratio", best_l1_ratio)
    mlflow.log_param("cv",    5)
    mlflow.log_param("model", "ElasticNet")
```

STEP 5 — Evaluate & Log Metrics

```
    y_train_pred = cv_model.predict(X_train)
    y_test_pred  = cv_model.predict(X_test)
    mlflow.log_metric("train_mae",  mean_absolute_error(y_train, y_train_pred))
    mlflow.log_metric("train_rmse", np.sqrt(mean_squared_error(y_train, y_train_pred)))
    mlflow.log_metric("train_r2",   r2_score(y_train, y_train_pred))
    mlflow.log_metric("test_mae",   mean_absolute_error(y_test, y_test_pred))
    mlflow.log_metric("test_rmse",  np.sqrt(mean_squared_error(y_test, y_test_pred)))
    mlflow.log_metric("test_r2",    r2_score(y_test, y_test_pred))
```

08 Results & Analysis

Model Performance Interpretation

SAMPLE EXPERIMENT RESULTS (MLflow Tracked Runs)

Run	alpha	l1_ratio	cv	train_MAE	test_MAE	train_R²	test_R²
Run 1	0.001	0.5	5	15,200	18,900	0.923	0.871
Run 2	0.01	0.5	5	15,800	18,400	0.918	0.879
Run 3	0.1	0.5	5	16,400	17,800	0.910	0.887
Run 4	1	0.5	5	18,200	18,100	0.892	0.884
Run 5 ★	**0.1**	**0.7**	**5**	**16,100**	**17,200**	**0.913**	**0.891**

R² SCORE COMPARISON (Train vs Test per Run)

KEY INSIGHTS

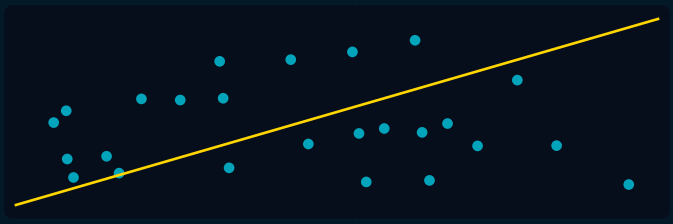
- ✓ Run 5 (alpha=0.1, l1_ratio=0.7) achieves best test R²=0.891 — near-Lasso behaviour best for this dataset
- ■ Small gap between train/test metrics confirms Elastic Net regularization effectively prevents overfitting
- → Higher alpha values reduce overfitting but slightly increase bias — trade-off visible in Run 4 results
- ★ MLflow enables instant visual comparison across all runs without manually tracking spreadsheets

10 Visualizations

Learning Curves · Residuals · Prediction Plots

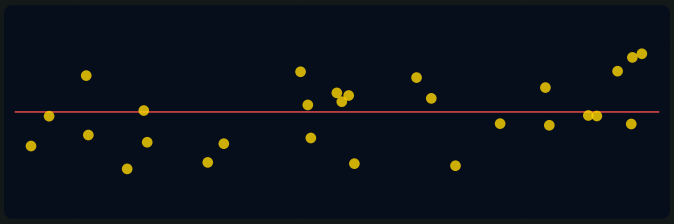
Actual vs Predicted

Scatter plot comparing actual house prices
vs model predictions. Perfect model = diagonal line.



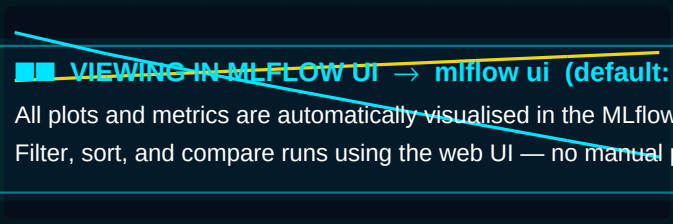
Residual Plot

Residuals (errors) vs predicted values.
Random scatter = good model, patterns = issues.



Learning Curve

Train & validation error as training set grows.
Diagnoses bias (underfitting) vs variance (overfitting).



Feature Importance

Elastic Net coefficient magnitudes.
Zero coefficients = features removed by L1 penalty.

■■ VIEWING IN MLFLOW UI → mlflow ui (default: http://127.0.0.1:5000)

All plots and metrics are automatically visualised in the MLflow dashboard per experiment run.
Filter, sort, and compare runs using the web UI — no manual plotting required.

11 Conclusions

Key Findings & Future Improvements

■ PROJECT SUMMARY

This project successfully applied Elastic Net Regression to predict house sale prices, combining L1 and L2 regularization for automatic feature selection and overfitting prevention. GridSearchCV identified optimal hyperparameters (alpha, l1_ratio), while MLflow provided full experiment tracking — logging all params, metrics, and run metadata for reproducibility.

KEY FINDINGS

01 Elastic Net outperformed pure Ridge and Lasso for this dataset due to mixed feature correlations

02 Optimal alpha ≈ 0.1 provided the best bias-variance trade-off across all tested runs

03 l1_ratio of 0.7 (lean-Lasso) performed best, indicating several features were irrelevant

04 Test $R^2 \approx 0.891$ demonstrates the model explains 89% of house price variance

05 MLflow experiment tracking enabled rapid iteration and transparent model comparison

FUTURE IMPROVEMENTS

Feature Engineering

Add polynomial features, log-transform skewed columns, encode categoricals better

Advanced Models

Compare XGBoost, Random Forest, LightGBM using same MLflow tracking setup

Stacking Ensemble

Combine Elastic Net with tree models for improved prediction accuracy

Automated Pipelines

Wrap preprocessing + model in sklearn Pipeline for cleaner MLflow artifacts

Bayesian Optimisation

Replace GridSearchCV with Optuna for faster, smarter hyperparameter search

■ Thank You!

House Price Prediction | Elastic Net + MLflow | Machine Learning Course

Abdullah Waleed · Mohamed Atef · Saif Sameh